

UNIT-II: Functions — Python Programming Lab Manual

Note: Each experiment must include Aim, Algorithm, Program, Output (blank for students to fill), and Result. Space has been left intentionally for professional formatting and signatures.

Experiment 7: Function with Multiple Return Values

Aim:

Design a function that processes inputs and returns multiple values at once.

Algorithm:

1. Define a function `analyze_numbers(a, b)`.
2. Compute sum, difference, product, and quotient (float) of the two numbers.
3. Return all computed values as a tuple.
4. In main, read two numbers from the user, call the function, unpack the results, and display them.

Program (Python):

Exp-7: Multiple return values

```
def analyze_numbers(a: float, b: float):  
    s = a + b  
    d = a - b  
    p = a * b  
    q = a / b if b != 0 else None  
    return s, d, p, q  
  
if __name__ == "__main__":  
    a = float(input("Enter first number: "))  
    b = float(input("Enter second number: "))  
    s, d, p, q = analyze_numbers(a, b)  
    print("Sum:", s)  
    print("Difference:", d)  
    print("Product:", p)  
    print("Quotient:", q)
```

Output:

Result:

Experiment 8: Function Using Default Arguments

Aim:

Illustrate default (optional) arguments in function definitions.

Algorithm:

1. Define a function power(base, exp=2) where exp defaults to 2.
2. If only the base is supplied, compute the square; otherwise, compute base raised to exp.
3. In main, accept user input for base and (optionally) exponent and display the result.

Program (Python):

Exp-8: Default arguments

```
def power(base: float, exp: int = 2) -> float:
    result = 1
    for _ in range(exp):
        result *= base
    return result

if __name__ == "__main__":
    base = float(input("Enter base: "))
    choice = input("Provide exponent? (y/n): ").strip().lower()
    if choice == 'y':
        exp = int(input("Enter exponent: "))
        print("Result:", power(base, exp))
    else:
        print("Result:", power(base))
```

Output:

Result:

Experiment 9: Find the Length of a String Without Using Library Functions

Aim:

Compute the length of a given string without using built-in length functions.

Algorithm:

1. Read a string from the user.
2. Initialize a counter to 0.
3. Iterate through each character and increment the counter.
4. Display the counter as the length.

Program (Python):

Exp-9: Length of string without built-ins

```
text = input("Enter a string: ")
count = 0
for _ in text:
    count += 1
print("Length:", count)
```

Output:

Result:

Experiment 10: Check if Substring is Present in a Given String

Aim:

Determine whether a pattern (substring) occurs in a string (without using high-level helpers like `.find()`).

Algorithm:

1. Read main string `s` and substring `sub`.
2. If `len(sub) == 0`, treat as present.
3. For each index `i` from 0 to `len(s) - len(sub)`, compare characters one by one.
4. If all characters match for some `i`, report **Found**; otherwise **Not Found**.

Program (Python):

```
# Exp-10: Manual substring search

s = input("Enter main string: ")
sub = input("Enter substring to search: ")

found = False
n, m = 0, 0
for _ in s: n += 1
for _ in sub: m += 1

if m == 0:
    found = True
else:
    for i in range(n - m + 1):
        match = True
        for j in range(m):
            if s[i + j] != sub[j]:
                match = False
                break
        if match:
            found = True
            break

print("Found" if found else "Not Found")
```

Output:

Result:

Experiment 11: List Operations — Addition, Insertion, and Slicing

Aim:

Demonstrate basic list operations: adding elements, inserting at a position, and slicing.

Algorithm:

1. Start with an initial list.
2. Append a new element (addition).
3. Insert an element at a specific index (insertion).
4. Display various slices of the list.

Program (Python):

```
# Exp-11: List addition, insertion, slicing
```

```
items = list(map(int, input("Enter integers (space-separated): ").split()))
```

```
x = int(input("Enter an element to append: "))
items.append(x)
```

```
idx = int(input("Enter index to insert at: "))
y = int(input("Enter element to insert: "))
if 0 <= idx <= len(items):
    items.insert(idx, y)
else:
    print("Invalid index; skipping insertion")
```

```
print("List after operations:", items)
print("Slice [0:3]:", items[0:3])
print("Slice [::2] (every 2nd):", items[::2])
print("Slice [-3:]:", items[-3:])
```

Output:

Result:

Experiment 12: Perform Any Five Built-in Functions on a List

Aim:

Use at least five built-in functions on a list and observe their effects/returns.

Algorithm:

1. Read a list of numbers.
2. Apply five built-ins (len, sum, min, max, sorted).
3. Display results; also demonstrate in-place reverse.

Program (Python):

Exp-12: Five built-ins on a list

```
nums = list(map(float, input("Enter numbers (space-separated): ").split()))

print("Count (len):", len(nums))
print("Sum (sum):", sum(nums))
print("Minimum (min):", min(nums) if nums else None)
print("Maximum (max):", max(nums) if nums else None)
print("Sorted copy (sorted):", sorted(nums))

nums.reverse()
print("List after reverse():", nums)
```

Output:

Result:
